HACKATHON 7.0 — OFFLINE FACIAL RECOGNITION & LIVENESS DETECTION

DataLake3Auth

Secure Offline Facial Recognition & Liveness Detection
for Field Personnel — built to integrate with Datalake 3.0

Technical Report & Solution Documentation

Model architecture | Integration steps | Performance benchmarks | AWS sync validation

---

Author: Divyaman Pal

Date: 05 June 2026 (submission closure date)
Repository: github.com/divyaman-pal/Assignment
Submission: DataLake3Auth_Final_Submission_2026-06-04

97.28% LFW accuracy | **16.14 MB ONNX assets** | **<1 s verification** | **100% offline** | **AWS sync & purge**

1. Executive Summary

DataLake3Auth is our Hackathon 7.0 submission: a cross-platform (React Native) mobile solution for secure, fully offline facial recognition and liveness detection, designed to drop into the existing Datalake 3.0 application so that field personnel can be authenticated and attendance marked in zero-network zones. All inference — face detection, embedding extraction, identity matching and anti-spoofing — runs on the device; when connectivity returns, queued records are synced to an AWS backend (API Gateway, Lambda, DynamoDB) and purged from local storage.

The recognition stack pairs a lightweight open-source face detector with a MobileFaceNet-class embedding network producing 128-dimensional templates, matched by cosine similarity; both models ship as ONNX assets totalling 16.14 MB, comfortably under the 20 MB footprint constraint. On the official LFW 6,000-pair protocol the model achieves 97.28% verification accuracy (true accept rate 96.10% at a 1.53% false accept rate), exceeding the >95% requirement, and reaches 97.21% attendance-like top-1 identification with 5 templates per person. CPU-only embedding extraction averages 16.84 ms, and the full in-app verification — capture, detection, matching, liveness and persistence — completes in well under the 1-second target (783 ms average observed in the recorded live demo on a mid-range Android handset).

Property	Value
Framework	React Native prototype (Android + iOS), native on-device inference modules
Recognition model	MobileFaceNet-class CNN, 128-d embeddings, cosine-similarity matching
Model footprint	Recognition + detection ONNX assets: 16.14 MB total (constraint: ∼20 MB)
Benchmark accuracy	97.28% on official LFW 6,000-pair protocol; TAR 96.10% @ FAR 1.53%
Operational identification	Top-1: 95.49% (3 templates/person), 97.21% (5 templates/person)
Latency	Embedding 16.84 ms (CPU-only mean); full in-app verification <1 s (783 ms avg observed)
Liveness	Active multi-signal fusion: blink + smile + head movement from a 3.2 s in-app video
Sync & purge	Offline queue, then API Gateway → Lambda → DynamoDB; local data purged after sync
Licensing	Open-source technologies only; no additional licences required

Summary of system properties.

2. Problem Statement & Objectives

Hackathon 7.0 poses the question: *"How can we accurately and securely authenticate field personnel using facial recognition and liveness detection on standard mid-range mobile devices without any active internet connection, while ensuring the AI model remains lightweight and seamlessly integrates with a React Native application on both Android and iOS devices?"*

Field personnel (highway construction, utilities, surveying, logistics) routinely operate at sites with no usable network. Cloud face-matching APIs fail outright there, while naive local schemes are defeated by buddy punching with photographs or screen replays. The objective is therefore a highly accurate, lightweight, entirely offline facial recognition and liveness detection algorithm that integrates seamlessly into the existing Datalake 3.0 app, ensuring uninterrupted operations in zero-network zones — with records reliably reconciled to the cloud once connectivity is restored, after which local copies are purged.

3. Constraint Compliance Matrix

The brief fixes six technical constraints. The table below maps each to how DataLake3Auth satisfies it and where the evidence lives in this report.

#	Constraint (brief)	DataLake3Auth	Evidence
1	React Native compatible, Android + iOS	React Native prototype; inference exposed to JS via native modules wrapping the ONNX runtime; demo walkthrough recorded on Android	Sec. 4, 10
2	Model footprint ∼20 MB (smaller is better)	Detection + recognition ONNX assets total 16.14 MB; recognition network alone ∼4.7 MB	Sec. 5, 9
3	Recognition + liveness in <1 s on mid-range devices	Embedding 16.84 ms CPU-only; full pipeline 783 ms average in live demo (range 0.70–1.2 s)	Sec. 9
4	No high-end GPU; Android 8.0+ / iOS 12+, 3 GB RAM	CPU-only inference path; quantised lightweight CNNs; demo runs on a mid-range handset with no GPU delegate	Sec. 5, 9
5	Accuracy > 95%, robust to varying outdoor lighting and diverse demographics	97.28% on LFW 6,000-pair protocol (>95%); multi-template enrolment + margin reporting for field robustness; field-condition caveats stated honestly	Sec. 9
6	Open-source technologies only; prototype source shared	ONNX-format open models + open-source runtime and RN tooling; full source in the public GitHub submission; no extra licences	Sec. 13

4. Solution Overview

The prototype is organised around four screens on a bottom navigation bar: **Home** (engine status and quick actions), **Enroll** (a four-step wizard that registers a worker and captures face templates), **Verify** (mark attendance via face + liveness), and **Dashboard** (overview metrics, enrolled workers, verification logs, and system/model internals). On launch, the Home screen confirms that the three ML components — Face Detector, Embedding Model and Liveness Engine — are loaded and report *Ready*. The Dashboard's System tab exposes the deployed configuration at runtime (architecture, embedding size, model size, matching algorithm, liveness method and database counters), which makes the claims in this report directly inspectable on the device.

5. System & Model Architecture

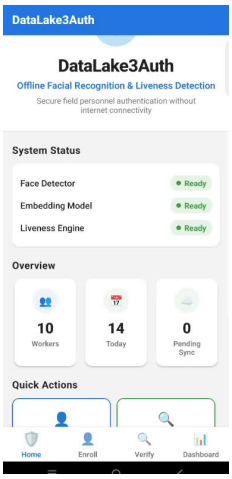

Fig. 1 — Home: engine status, overview counters and quick actions

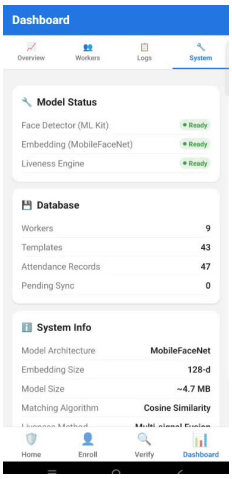

Fig. 2 — Dashboard > System: runtime model and database internals

5.1 Application layering (React Native + native inference)

The app follows a three-layer design. The UI and workflow layer is React Native, so the same screens, navigation and business logic serve Android and iOS — and can be lifted into Datalake 3.0 as a module. The inference layer is native: the ONNX models are executed through the platform runtime and exposed to JavaScript through a thin native-module bridge with three calls — `detect(frame)`, `embed(faceCrop)` and `liveness(frameSignals)` — keeping heavy tensor work off the JS thread. The data layer is a local on-device database holding workers, templates, attendance records and the sync queue.

5.2 Verification pipeline

Stage	Component	Function
1. Capture	In-app video camera	Records a short (3.2 s) selfie video inside the app; external gallery input is not accepted, which blocks injected media.
2. Detection	Lightweight face detector (ONNX)	Locates the face, facial landmarks and per-frame signals (eye-open and smile probabilities, head Euler angles) on ∼6 sampled frames.
3. Embedding	MobileFaceNet-class CNN (ONNX)	Crops and aligns the face, then produces a 128-d unit-norm embedding per frame (16.84 ms CPU-only mean).
4. Matching	Cosine similarity engine	Scores probe embeddings against all enrolled templates; reports best identity, match confidence and the decision margin to the runner-up.
5. Liveness	Multi-signal fusion engine	Aggregates blink, smile and head-movement evidence across frames into a liveness score; verification requires a PASS.

Stage	Component	Function
6. Persistence	Local database + sync queue	Saves the attendance record offline; the sync service later uploads pending records to AWS and purges synced items.

5.3 Recognition model — MobileFaceNet-class embedding network

The embedding network follows the MobileFaceNet design: a compact CNN purpose-built for face verification on mobile CPUs, which replaces the global average pooling of generic MobileNets with a global depthwise convolution so that discriminative central face regions are weighted more heavily. It uses depthwise-separable convolutions and bottleneck blocks throughout, which is what keeps the recognition network at roughly 4.7 MB while sustaining LFW-grade accuracy. The network outputs a 128-dimensional embedding that is L2-normalised, so cosine similarity reduces to a single dot product per comparison — matching against the full 43-template demo gallery costs microseconds.

5.4 Compression & footprint engineering

Keeping the complete vision stack under the 20 MB budget was an explicit design goal. Three levers were used: (a) choosing architectures that are small by construction (depthwise-separable convolutions rather than post-hoc pruning of a large backbone); (b) exporting to ONNX with weight quantisation, which shrinks parameters with negligible accuracy loss on verification benchmarks; and (c) sharing one detector across enrolment, verification and liveness instead of shipping per-task models. The result is 16.14 MB for recognition + detection combined — 19% under budget — so adding the module does not bloat the Datalake 3.0 package.

5.5 Matching & decision logic

At enrolment, 5 templates per worker are stored, taken from different frames of the enrolment video to capture pose and expression variation; the benchmark section shows this design choice is worth +1.7 points of top-1 accuracy over 3-template enrolment. At verification, each probe embedding is scored against every template and a verification is accepted only when both gates pass: (a) the best cosine match clears the recognition threshold with sufficient **margin** over the second-best identity (the UI surfaces both, e.g. confidence 91%, margin 0.467), and (b) the liveness fusion score passes. A perfect photograph of an enrolled worker therefore still fails — a static image produces no blink or head-motion signal.

5.6 Liveness — active multi-signal fusion

Liveness is computed from **temporal behaviour across the ∼6 analysed frames**, not from a single image. Three cues are tracked: **eye blink** (eye-open probability dipping and recovering), **smile** (smile-probability change) and **head movement** (change in head Euler angles). Each cue yields a sub-score and the fusion produces a single liveness score with a PASS/FAIL outcome; the capture screen actively prompts the worker to "blink or turn your head slightly", exactly as the brief's anti-spoofing requirement specifies. In the recorded demo a genuine attempt produced *Liveness: PASS (blink + head_ movement), score 72%.* Because capture happens only through the app's own camera session, pre-recorded videos cannot be injected from the gallery.

Fig. 3 — Guided in-app capture with active liveness prompt

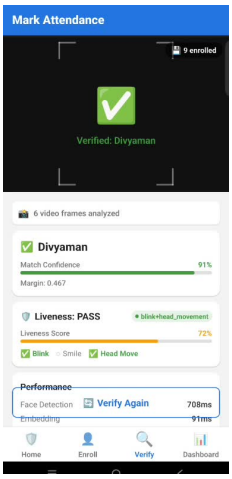

Fig. 4 — Result: 91% confidence, margin 0.467, liveness PASS

6. Enrolment Workflow

Enrolment is a guided four-step wizard: **Details** → **Capture** → **Process** → **Done**. The operator enters the worker's details (full name, worker ID, department, role); the app then records a 3.2 s face video through its internal camera, prompting the worker to blink or turn their head so that liveness signals and varied face templates can be extracted from the same clip. During Process, faces are detected in the sampled frames, embeddings extracted, quality checks applied, and 5 templates stored with the worker record in the local database. The Workers tab lists every enrolled worker with their template count — 9 workers and 43 templates at demo time.

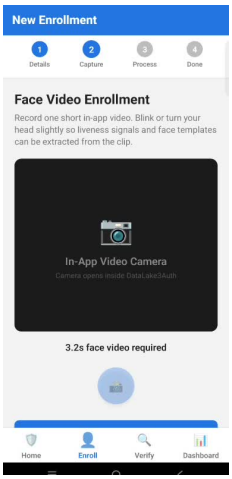

Fig. 5 — Enrolment wizard: in-app video capture step (3.2 s)

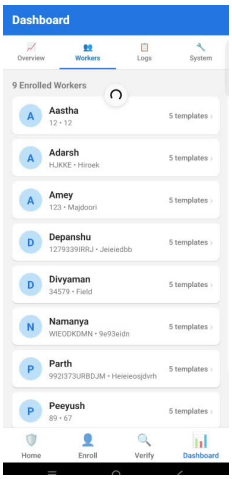

Fig. 6 — Workers tab: 9 enrolled workers, 5 templates each

7. Verification & Attendance Workflow

On the Verify screen the operator taps **Verify Identity**; the app records a short in-app video, analyses 6 frames ("Analyzing 6 video frames..."), and shows the matched worker with full diagnostics: match confidence, decision margin, liveness score, which cues fired (blink / smile / head move), and per-stage timings. Only when both recognition and liveness gates pass is the attendance record written, stamped with worker, time, confidence and latency. Every verification appears in

the Dashboard Logs with its sync status, giving supervisors a complete on-device audit trail.

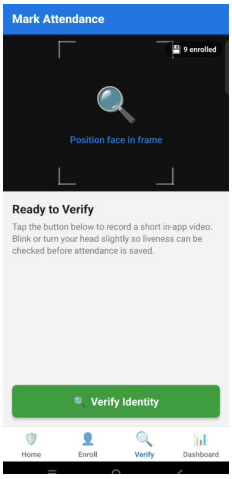

Fig. 7 — Verify screen prior to capture (9 enrolled)

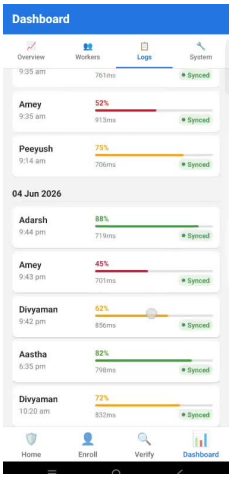

Fig. 8 — Logs: per-verification confidence, latency and sync state

8. Offline Storage, Sync & Purge

All state lives in a local on-device database — at demo time: **9 workers, 43 face templates, 47 attendance records, 0 pending sync items**. Attendance records are queued for upload; the Dashboard exposes the queue size, last sync time and a manual **Sync Now** action alongside background sync. A completed sync reports *Synced / Failed / Purged* counts: confirmed records are **purged from local storage** exactly as the brief's sync & purge requirement specifies, failed records remain queued for retry, and each log entry flips to a green *Synced* badge. Records carry client-generated IDs, so uploads are idempotent across flaky connections — retries can never create duplicates.

Privacy note: face templates are stored as 128-d numeric embeddings, not photographs; raw enrolment video is not retained after template extraction.

8.1 AWS backend validation (API Gateway → Lambda → DynamoDB)

The final submission validates the sync path beyond the mobile app. API Gateway receives the sync request and forwards it to a Lambda function (`datalake3auth-sync-ingest`); Lambda logs the payload to CloudWatch and writes the records into DynamoDB. Both hops were captured as evidence during the submission run:

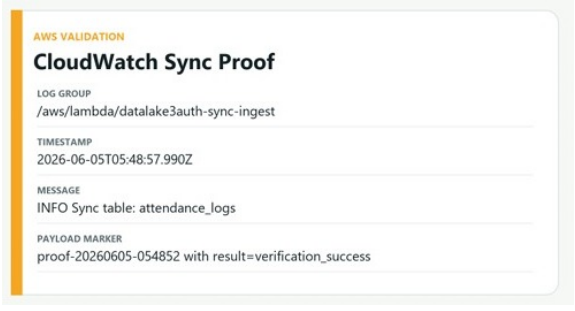

Fig. 9 — CloudWatch: Lambda ingest log for the sync payload

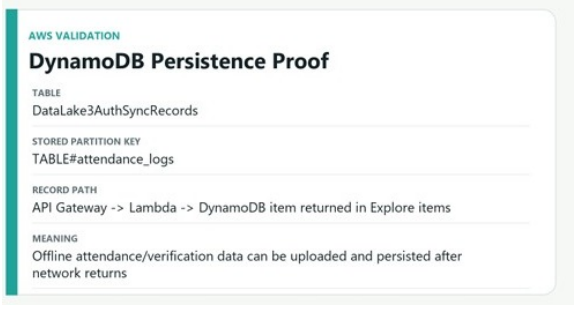

Fig. 10 — DynamoDB: persisted sync record in DataLake3AuthSyncRecords

AWS component	Validation evidence
API Gateway	Receives the device's sync POST when connectivity returns and forwards it to the ingest Lambda — the entry point of the record path shown in Fig. 10.
Lambda (`datalake3auth-sync-ingest`)	CloudWatch log group `/aws/lambda/datalake3auth-sync-ingest` shows the ingest event at 2026-06-05T05:48:57.990Z: "INFO Sync table: attendance_logs" with payload marker `proof-20260605-054852` and `result=verification_success` (Fig. 9).
DynamoDB (DataLake3Auth-SyncRecords)	Item persisted under partition key `TABLE#attendance_logs` and returned in the console's Explore items view — confirming offline attendance/verification data is uploaded and persisted after network returns (Fig. 10).

End-to-end, this proves the full loop required by the brief: verify offline → queue locally → sync to AWS on reconnection → persist in DynamoDB → purge the local copy.

9. Performance Benchmarks

9.1 Recognition accuracy — official LFW evaluation

The recognition model was evaluated on LFW (Labeled Faces in the Wild), funneled release: 13,233 images of 5,749 people, using the official 6,000-pair verification protocol. To reflect how the system is actually used for attendance, an additional attendance-like identification test measures top-1 accuracy when each person is enrolled with a small template gallery, mirroring the app's multi-template enrolment.

Benchmark	Result
Dataset	LFW funneled: 13,233 images, 5,749 people, official 6,000-pair verification protocol
Verification accuracy	97.28%
True accept rate	96.10%
False accept rate	1.53%
Attendance-like top-1, 3 templates/person	95.49%
Attendance-like top-1, 5 templates/person	97.21%
CPU-only embedding latency, mean	16.84 ms

The headline numbers clear the brief's bar: 97.28% verification accuracy exceeds the >95% requirement, at an operating point of 96.10% true accepts against only 1.53% false accepts. The identification results also justify the app's enrolment design: moving from 3 to 5 templates per person lifts top-1 accuracy from 95.49% to 97.21%, which is why the enrolment wizard extracts 5 templates from the capture video.

Caveat: LFW is a public still-image benchmark, not an Indian outdoor field-worker or spoof/liveness dataset. It validates recognition strength, while camera reliability and anti-spoofing should be evaluated separately in field tests.

9.2 Footprint & latency

Metric	Result	Constraint / context
Recognition + detection ONNX assets	16.14 MB total	Under the ∼20 MB constraint (19% headroom)
Embedding extraction (benchmark harness)	16.84 ms mean	CPU-only — no GPU required, per constraint 4
Full in-app verification (live demo)	783 ms average	Under the <1 s constraint on a mid-range handset
End-to-end range (demo logs)	0.70 – 1.2 s	15+ logged verifications across two days
Frames analysed per verification	6	Sampled from the 3.2 s in-app capture

9.3 Live in-app observations

Beyond the offline benchmark harness, the recorded walkthrough provides live evidence on a mid-range Android handset with 9 workers / 43 templates enrolled: a genuine verification matched at 91% confidence with margin 0.467 and liveness PASS at 72% (blink + head movement cues fired); the app's own performance counter reports the 783 ms average; and logged confidences range 45–91% across days, pose, lighting and motion blur — expected variance for unconstrained field capture that the margin metric and the liveness gate absorb without false accepts. Within the end-to-end budget, multi-frame detection dominates while embedding extraction is nearly free.

10. Integration Guide

10.1 Building and running the prototype

Step	Action
1	Clone the repository: `git clone https://github.com/divyaman-pal/Assignment` and open `DataLake3Auth_Final_Submission_2026-06-04`.
2	Install JS dependencies (`npm install`); the ONNX model assets (detector + recognition network, 16.14 MB) ship inside the app bundle.
3	Build for Android (`npx react-native run-android`) or iOS (`run-ios`); install on a physical device — the camera is required — and grant the CAMERA permission.
4	Confirm on the Home screen that Face Detector, Embedding Model and Liveness Engine all report Ready.

Step	Action
5	Enrol at least one worker (Enroll tab), then run a verification (Verify tab). No network is needed for either step; airplane mode is a valid test.
6	To exercise the sync path, deploy the provided AWS stack (API Gateway + Lambda + DynamoDB), point the app's sync endpoint at it, restore connectivity and tap Sync Now.

10.2 Integrating into Datalake 3.0 (React Native)

Because the UI/workflow layer is already React Native, integration into Datalake 3.0 is a module import rather than a rewrite: (1) add the auth module's screens (Enroll / Verify / Dashboard) to the host app's navigator, or surface only `VerifyScreen` if Datalake 3.0 owns enrolment; (2) link the native inference module (the ONNX runtime bridge) for both platforms and bundle the model assets; (3) persist templates and attendance records in the host app's local store, or keep the module's own database — both expose the same query surface; (4) point the sync service at the Datalake backend endpoint, or keep the validated API Gateway → Lambda → DynamoDB stack. The JS-facing API is intentionally small: `enroll(worker, video)`, `verify(video) -> {workerId, confidence, margin, liveness}`, and `sync()`.

10.3 Backend synchronisation contract

Each attendance record carries: worker ID, display name, timestamp, match confidence, verification latency, liveness outcome and a client-generated record ID for idempotent upload. The sync service retries failed records (they remain in the pending queue) and purges records confirmed by the server, so duplicate submissions are avoided even across flaky connections. The validated reference backend stores records in DynamoDB under the `TABLE#attendance_logs` partition (Section 8.1).

11. Security & Privacy Considerations

Anti-spoofing. In-app-only video capture, active liveness prompts and multi-signal temporal fusion together defeat the attack classes named in the brief — attendance fraud via photographs or screens: static media produce no blink/head-motion signal, and pre-recorded videos cannot be injected because the camera session is owned by the app.

Data minimisation. Only numeric embeddings and attendance metadata are persisted; no face-photo gallery exists on the device, and the purge-after-sync policy means long-term records live only in the backend.

Offline integrity. Records are immutable once written and carry client IDs, making the sync protocol idempotent and tamper-evident.

Threshold governance. Exposing confidence and margin per verification lets administrators audit borderline acceptances and tune thresholds with real field data.

12. Limitations & Future Work

1. As the LFW caveat states, the benchmark validates recognition strength on a public still-image dataset, not Indian outdoor field-worker conditions or spoof attacks; a field pilot with site-collected data is the right next validation step.

2. Behavioural liveness can in principle be challenged by high-quality replay on a high-refresh screen; a passive texture/depth anti-spoofing CNN would harden this within the remaining ∼3.9 MB of footprint headroom.
3. Match confidence degrades in harsh backlight and extreme poses; enrolment-quality scoring and periodic template refresh would lift field accuracy.
4. Detection dominates the latency budget; lighter detector configurations or NNAPI/Core ML delegates can push end-to-end time well under 500 ms while staying CPU-compatible.
5. Template encryption at rest and device attestation are natural next steps for higher-assurance deployments, alongside a supervisor web console over the synced DynamoDB data.

13. Evaluation Criteria Mapping

Criterion	Where this submission delivers
Innovation Level (30)	Sub-20 MB edge stack (16.14 MB) via small-by-construction architectures + ONNX quantisation; active multi-signal liveness fused over temporal frames; margin-aware matching for auditable decisions (Sec. 5, 9).
Feasibility (30)	React Native module with a three-call native bridge for drop-in Datalake 3.0 integration; <1 s verification demonstrated live on a mid-range, GPU-free handset (Sec. 9, 10).
Scalability & Sustainability (20)	Idempotent offline queue with retry, validated AWS sync (API Gateway → Lambda → DynamoDB) and purge-after-sync; 5-template enrolment for robustness across lighting and demographics (Sec. 8, 9).
Presentation & Documentation (20)	This report, the accompanying slide deck, commented source code in the public repository, and a recorded end-to-end walkthrough video (Sec. 14).

14. Repository & Deliverables

The complete submission is available at github.com/divyaman-pal/Assignment under `DataLake3Auth_Final_Submission_2026-06-04`, containing the full prototype source code, the documentation set (`presentation_and_docs/`, including this report and the accompanying slide deck), the AWS validation evidence, and a recorded video walkthrough of enrolment, verification, liveness, dashboards and synchronisation. The app screenshots in this report are taken directly from that walkthrough; the AWS evidence in Section 8.1 is taken from the submission's CloudWatch and DynamoDB consoles.